A2 – S4 – 2025/2026 – II.2415
Advanced Algorithms and Programming
Semester Project

LAB_4

Recursive Algorithms

Session Objectives

By the end of this session, you will be able to:

1. Identify problems with recursive structure
2. Implement recursive traversals of nested data
3. Apply divide-and-conquer strategies
4. Convert between recursive and iterative solutions with explicit stacks
5. Analyze recursion depth and stack memory implications

Core Exercises

Exercise 1: Recursive Comment Thread Traversal
Objective: Implement recursive traversal of nested comment threads.
Social media comments form nested structures: users reply to comments, creating deep conversational threads. Understanding recursion is essential for processing this nested data.
Requirements:

1. Create a CommentNode structure containing:
    - comment_id: unique identifier
    - user_id: author identifier
    - content: string (limited to 100 chars)
    - timestamp: time posted
    - likes: integer
    - replies: list of CommentNode objects (nested structure)
2. Implement recursive traversal functions:
    - display_thread(comment, level): recursively display comment and all replies with indentation (2 spaces per level)
    - count_total_comments(comment): recursively count all comments in a thread (including nested replies)
    - total_likes(comment): recursively sum all likes in a thread (comment + all replies)
    - find_deepest_reply(comment): recursively find maximum nesting depth
3. Implement search functionality:
    - search_by_user(user_id, comment): recursively collect all comments by a specific user
    - contains_keyword(keyword, comment): recursively check if any comment contains keyword
4. Implement deletion with cascade:
    - delete_comment(comment_id, thread): recursively remove a comment and all its replies

- Return new thread structure (without deleted comment)

Examples:
Consider a post with the following comment thread structure:
Comment 101 (Alice): "This recipe looks amazing!"
├── Reply 201 (Bob): "I tried it last night!"
│ └── Reply 301 (Alice): "What did you think?"
│ └── Reply 401 (Bob): "It was delicious!"
└── Reply 202 (Charlie): "Can I use olive oil instead?"
└── Reply 302 (Alice): "Yes, that works too!"
Visual representation of the nested structure:
Level 0: Comment 101 (Alice)
Level 1: Reply 201 (Bob)
Level 2: Reply 301 (Alice)
Level 3: Reply 401 (Bob)
Level 1: Reply 202 (Charlie)
Level 2: Reply 302 (Alice)
Complexity Analysis Questions:

1. What is the time complexity of traversing an entire comment thread with n total comments?
2. How does recursion depth relate to the comment nesting structure? What happens with very deep threads (1000+ levels)?
3. Compare memory usage between recursive and iterative stack-based implementations

Exercise 2: Recursive Content Aggregation with Divide & Conquer
Objective: Apply divide-and-conquer recursion to process engagement metrics efficiently. Many analytics can be computed using recursive splitting strategies.
Requirements:

1. Create a Post structure:
   - post_id: unique identifier
   - user_id: author
   - content_preview: string
   - timestamp: time posted
   - likes: integer
   - comments: integer
   - shares: integer
   - engagement_score: computed as (likes × 1 + comments × 2 + shares × 3)
2. Implement divide-and-conquer analytics on arrays of posts:
   Part A: Maximum Engagement
   - max_engagement(posts, left, right): recursively find post with highest engagement score
     - Base case: single post → return its score
     - Recursive case: split array in half, find max in each, return larger

   Part B: Total and Average
   - sum_engagement(posts, left, right): recursively sum all engagement scores
   - average_engagement(posts, left, right): use sum_engagement to compute average

Part C: Count by Threshold

- count_above_threshold(posts, left, right, threshold): recursively count posts with engagement > threshold

Part D: Merge Sort by Engagement

- merge_sort_by_engagement(posts, left, right): recursively sort posts by engagement score
- merge(posts, left, mid, right): merge two sorted halves

3. Implement peak hours analysis on a single post's hourly data:
    - hourly_likes: array of 24 integers (likes per hour)
    - find_peak_hour(likes, left, right): recursively find hour with maximum likes
        - Compare middle element with neighbors to determine which half contains peak

Examples:
Posts array:

- Post1: engagement 150
- Post2: engagement 320
- Post3: engagement 95
- Post4: engagement 280

max_engagement(posts, 0, 3) → Post2 (320)
sum_engagement(posts, 0, 3) → 845
average_engagement(posts, 0, 3) → 211.25
count_above_threshold(posts, 0, 3, 200) → 2 (Post2, Post4)
Hourly likes: [5, 8, 12, 25, 30, 28, 15, 10, ...]
find_peak_hour(likes, 0, 23) → hour 4 (30 likes)
Complexity Analysis Questions:

1. What is the time complexity of recursive max_engagement? Prove it.
2. Compare recursive merge sort (O(n log n)) with iterative insertion sort ($O(n^2)$) for sorting 10,000 posts
3. What is the recursion depth for merge sort on n elements?
4. For find_peak_hour, why can we use binary search-style recursion? What property must the array satisfy?

Exercise 3: Converting Recursion to Iteration with Explicit Stack
Objective: Learn when recursion is appropriate and how to convert to iteration using explicit stacks when stack depth is a concern.
Requirements:

1. Implement recursive comment thread flattener:
    - flatten_recursive(comment): return a list of all comments in the thread in order (depth-first)
    - Example: [Comment1, Reply1, SubReply1, Reply2]
2. Implement iterative version with explicit stack:
    - flatten_iterative(comment): produce same result using your own stack
    - Store (comment, state) where state tracks processing progress
    - Define states:
        - STATE_START: just pushed, haven't processed replies
        - STATE_REPLIES_DONE: finished processing replies
3. Implement tail recursion conversion for counting:

    - count_comments_tail(comment, accumulator): tail recursive version that passes running total
    - count_comments_loop(comment): convert to while loop
4. **Performance comparison**:
    - For thread depths: 5, 10, 50, 100, 500
    - Estimate/measure stack memory usage for recursive vs iterative
    - Identify at what depth recursion might cause stack overflow

Examples:

- Same nested comment structure from Exercise 1
  flatten_iterative(Comment1) output:
  [Comment1, Reply1, SubReply1, Reply2]
- Stack simulation visualization:
  Step 1: Push (Comment1, START)
  Step 2: Pop (Comment1, START) → add to result, push (Comment1, REPLIES_DONE), push replies in reverse order
  Step 3: Pop (Reply2, START) → add to result, push (Reply2, REPLIES_DONE)
  Step 4: Pop (Reply1, START) → add to result, push (Reply1, REPLIES_DONE), push SubReply1
  ...

Complexity Analysis Questions:

1. For a thread with depth d and total n comments, compare maximum stack size needed for:
    - Recursive implementation
    - Iterative with explicit stack
2. When would you prefer recursive version? When iterative?
3. How does tail recursion conversion reduce stack usage? Can all recursive functions be made tail recursive?
4. Explain the state machine approach—why is it necessary for functions that process nested structures?
5. In a production social network with user-generated nested comments, which approach would you choose and why?

Final Integration Question

Based on your previous work, design a feature: "Engagement Analytics with Nested Comments"

Combine:

- Set operations: find users who commented on multiple posts
- Doubly linked lists: navigate through top posts
- Priority queues: rank posts by engagement
- Recursive algorithms: process nested comment threads

Describe:

1. How these structures work together
2. Which operations would be recursive vs. iterative
3. How you'd handle potential stack overflow with deep threads
4. Performance considerations for millions of posts

General Instructions

Repository Setup
Create branch: LAB _04_RecursiveAlgorithms

File Structure:
Follow the same structure of LAB _03

Git Requirements:
Same as for LAB _03

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 10	**Note: 10**
• Each Team (of number BB) Upload at the **end of the session** a very short PDF document and name it **TBB_LAB4_AES.pdf**, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	• You finish all the implementation of all exercises, • You finalize the previous report and give it a new name **TBB_LAB4_BFM.pdf** and upload it before **Friday midnight of the same week**, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.